UNIT 1: Introduction to Recursion

1. What is Recursion?

Recursion is a programming technique where a function calls itself to solve a smaller version of the same problem. Instead of solving a large problem directly, recursion breaks the problem into smaller subproblems that are easier to solve.

Recursion is particularly useful when working with problems that have a natural hierarchical or similar structure, such as trees, graphs, and divide and conquer algorithms.

2. Key Components of Recursion

Every recursive function must contain two essential components:

- **Base Case:** The condition that stops the recursion. Without a base case, the function would call itself indefinitely.
- **Recursive Case:** The part where the function calls itself with a smaller or simpler input.

3. Example: Factorial Function

Mathematically:

```
n! = n × (n-1)!
```

Python implementation:

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n-1)
```

The function continues calling itself with smaller values until it reaches the base case (when n equals 0).

4. Recursive Execution Example

```
factorial(4)
= 4 * factorial(3)
= 4 * 3 * factorial(2)
= 4 * 3 * 2 * factorial(1)
= 4 * 3 * 2 * 1 * factorial(0)
= 24
```

5. The Call Stack

When recursion occurs, each function call is stored in a structure called the call stack. The most recent function call is executed first. Once the base case is reached, the stack unwinds and results

are returned step by step.

```
Call Stack Example:

factorial(4)
factorial(3)
factorial(2)
factorial(1)
factorial(0)
```

6. Recursive Data Structures

Recursive data structures are structures defined in terms of themselves. Examples include linked lists and trees.

- **Linked Lists:** Each node contains data and a reference to another node.
- **Trees:** Each node may contain references to child nodes which are themselves trees.

Example: Binary Tree Node in Python

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
```

7. Tree Traversal Using Recursion

Example: Inorder Traversal

```
def inorder(node):
    if node is None:
        return
    inorder(node.left)
    print(node.value)
    inorder(node.right)
```

8. Advantages of Recursion

- Simplifies complex problems.
- Natural for hierarchical structures like trees.
- Often leads to clearer and shorter code.

9. Disadvantages of Recursion

- Uses more memory due to the call stack.
- May lead to stack overflow if the recursion depth is too large.
- Sometimes slower than iterative solutions.

10. Recursion vs Iteration

```
Recursion                      Iteration
---------------------------------------------
Uses function calls            Uses loops
Uses call stack                Uses variables
Often simpler for trees        Often more memory efficient
```

11. Common Mistakes in Recursion

- Missing base case.
- Incorrect base condition.
- Recursive call does not move toward the base case.

12. Practice Exercise

- Write a recursive function to print numbers from n down to 0.
- Write a recursive function to compute the sum of numbers from 1 to n.
- Modify the factorial program to show each recursive call.